Basics Day 2 — Session 2 (Hours 5–8)

Python Programming (Basic) • Strings, Input, Conversion & Checkpoint 1

Session 2 Overview

Topics Covered Today

- Hour 5: String indexing, slicing, and `len()`
- Hour 6: String methods — normalize, search, replace
- Hour 7: Interactive input and type conversion
- Hour 8: Checkpoint 1 — fundamentals mini-assessment

Hour 5: String Indexing, Slicing, and `len()`

Learning Outcomes

- Access individual characters using index notation
- Extract substrings with slice syntax (`start:end`)
- Use `len()` to measure length and avoid out-of-range errors
- Perform basic membership checks with the `in` operator

> *Source:* `Basics/lessons/lecture/Day2_Hour1_Basics.md` *— Section 0)*

Strings Are Ordered Sequences

Think of Each Character as a Labeled Box

```
word = "Python"
#       P  y  t  h  o  n
# index 0  1  2  3  4  5
```

Indexing Starts at Zero — Not One

```
print(word[0])   # P
print(word[1])   # y
print(word[5])   # n
```

> "Index means position. Position counting starts at 0." *Source:* `Day2_Hour1_Basics.md` *— Section 3.2*

Negative Indexing

Count Backwards from the End

```
word = "Python"
print(word[-1])   # n  (last character)
print(word[-2])   # o
print(word[-6])   # P  (same as word[0])
```

When Is This Useful?

- File extensions: `filename[-3:]` → `"pdf"`
- Last character of user input
- End is more predictable than the beginning

> *Source: `Day2_Hour1_Basics.md` — Section 3.3*

Slicing Fundamentals

Syntax: `s[start:end]` — End Is Exclusive

```
word = "Python"
print(word[0:3])   # Pyt   (indexes 0, 1, 2)
print(word[2:5])   # tho
print(word[:4])    # Pyth  (start defaults to 0)
print(word[1:])    # ython (end defaults to len)
```

The One Rule to Memorize

> "`s[a:b]` **includes** `a`, **excludes** `b`"

If your slice looks one character short — check your end index.

> *Source:* `Day2_Hour1_Basics.md` *— Section 3.4*

`len()` and the `in` Operator

`len()` — Your Measuring Tape

```
message = "Hello, World!"
print(len(message))                    # 13
print(message[len(message) - 1])       # !  (last char, safe)
```

`in` — Boolean Membership Check

```
text = "Hello, Python learners!"
print("Python" in text)    # True
print("python" in text)    # False  ← case-sensitive!
```

Use `in` when the question is **yes/no**. Use `find()` when you need the **position**.

> *Source: `Day2_Hour1_Basics.md` — Sections 3.5–3.6*

Demo: Initials + Email Domain

Extract Initials from a Name

```
full_name = "Alice Smith"
first_initial = full_name[0]
space_index = full_name.find(" ")
last_initial = full_name[space_index + 1]
print(f"Initials: {first_initial}.{last_initial}.")
# Output: Initials: A.S.
```

Extract Domain from an Email

```
email = "student@example.com"
at_index = email.find("@")
domain = email[at_index + 1:]
print(f"Domain: {domain}")
# Output: Domain: example.com
```

> *Source: `Day2_Hour1_Basics.md` — Section 4, Demos A–B*

Lab: Username Builder

Time: 25–35 minutes

Requirements

1. Ask for first name and last name
2. Build username: `first initial + last name`
3. Normalize to lowercase; remove spaces in last name
4. Print username and its character count

Example Interaction

```
Enter first name: Alice
Enter last name: Van Halen
Username: avanhalen
Length: 9 characters
```

Reference Solution: Username Builder

```python
# Username Builder (Hour 5)
first_name = input("Enter first name: ").strip()
last_name  = input("Enter last name: ").strip()

if len(first_name) == 0 or len(last_name) == 0:
    print("Please provide both first and last name.")
else:
    first_initial = first_name[0].lower()
    last_clean    = last_name.lower().replace(" ", "")
    username      = first_initial + last_clean

    print(f"Username: {username}")
    print(f"Length: {len(username)} characters")
```

> *Source:* `Day2_Hour1_Basics.md` *— Section 5.3*

Common Pitfalls (Hour 5)

⚠️ **`IndexError` from empty input** — always check `len()` before indexing

⚠️ **Forgetting normalization** — `first[0] + last` gives `"AVan Halen"` not `"avanhalen"`

⚠️ **Off-by-one in slices** — remember: end index is excluded

Quick Fix Pattern

```
if len(name) > 0:
    print(name[0])
```

Quick Check (Hour 5)

1. What does `s[-1]` return?
2. If `s = "Python"`, what is `s[1:4]`?
3. Why is `len(s) - 1` the safe last index?
4. Is `"py" in "Python"` True or False — and why?

Answers: last char · `"yth"` · indexing starts at 0 · **False** (case-sensitive)

Hour 6: String Methods — Normalize, Search, Replace

Learning Outcomes

- Use `.lower()`, `.upper()`, `.strip()`, `.replace()`, `.find()`, `.count()`
- Explain why strings are **immutable** and why methods return new strings
- Choose `find()` vs `in` based on whether you need position or presence
- Clean messy user input with method chaining
- Detect keywords while handling capitalization variations

> *Source:* `Basics/lessons/lecture/Day2_Hour2_Basics.md` *— Learning Outcomes*

String Immutability

Strings Cannot Be Changed In-Place

```
message = "Hello"
message.upper()    # Returns "HELLO" — but does NOT change message!
print(message)     # Still prints "Hello"
```

Always Assign the Return Value

```
message = message.upper()
print(message)     # Now prints "HELLO"
```

> **Key rule:** String methods **always** return a new string. Assign the result or you lose it.

> *Source:* `Day2_Hour2_Basics.md` *— Core Concepts §1*

Case Transformation Methods

Method	Result
`.lower()`	all lowercase
`.upper()`	ALL UPPERCASE
`.title()`	Title Case
`.capitalize()`	First letter only

Example

```
user_input = "jAvA is GREAT"
print(user_input.lower())    # "java is great"
print(user_input.title())    # "Java Is Great"
```

> *Source:* `Day2_Hour2_Basics.md` *— Core Concepts §2*

Whitespace Management

Stripping Methods

```
dirty = "  hello  world  "
print(dirty.strip())    # "hello  world"  ← outer gone, inner stays!
print(dirty.lstrip())   # "hello  world  "
print(dirty.rstrip())   # "  hello  world"
```

Important: Strip Does NOT Remove Embedded Spaces

To remove internal spaces use `.replace()`:

```
no_spaces = dirty.strip().replace(" ", "")
print(no_spaces)   # "helloworld"
```

> *Source:* `Day2_Hour2_Basics.md` *— Core Concepts §3*

String Searching

Three Tools — Pick the Right One

```
text = "Python is powerful"

# Use 'in' for True/False only:
print("is" in text)          # True

# Use .find() when you need the position:
pos = text.find("is")
print(pos)                   # 7

# Use .count() for total occurrences:
print(text.count("o"))       # 2
```

Critical Detail

> `.find()` returns `-1` when not found — **not** `False` or `None`

```
```

String Replacement

`.replace(old, new)` — Returns a New String

```
text = "The cat sat on the mat"
result = text.replace("cat", "dog")
print(result)   # "The dog sat on the mat"
print(text)     # "The cat sat on the mat"  ← unchanged!
```

Case-Sensitive by Default

```
# "The" at start is NOT matched by "the"
fixed = text.lower().replace("the", "a")
print(fixed)    # "a cat sat on a mat"
```

> *Source: `Day2_Hour2_Basics.md` — Core Concepts §5*

Method Chaining

Combine Multiple Methods in One Expression

```
user_input = "  PYTHON IS AWESOME  "
clean = user_input.lower().strip().replace("awesome", "great")
print(clean)    # "python is great"
```

For Long Chains — Break for Readability

```
result = (user_input
    .lower()
    .strip()
    .replace("python", "java"))
```

> Always read chains **left to right** — each method acts on the result of the previous one.

> *Source: `Day2_Hour2_Basics.md` — Core Concepts §6*

Demo: Email Validation Helper

```python
email = "  JOHN.DOE@EXAMPLE.COM  "

# Step 1: Remove whitespace
email = email.strip()
print(f"After strip: '{email}'")    # 'JOHN.DOE@EXAMPLE.COM'

# Step 2: Lowercase
email = email.lower()
print(f"After lower: '{email}'")    # 'john.doe@example.com'

# Step 3: Check for @
if "@" in email:
    print("Valid format (contains @)")

# Step 4: Extract prefix
at_pos = email.find("@")
prefix = email[:at_pos]
print(f"Prefix: {prefix}")          # john.doe

# Step 5: Count dots
print(f"Total dots: {email.count('.')}")   # 2
```

Lab: Text Sanitizer

Time: 30 minutes

Goal

Build a text sanitizer that:

1. Strips leading/trailing spaces and converts to lowercase
2. Replaces multiple consecutive spaces with a single space
3. Counts occurrences of a given keyword (case-insensitive)

Expected Results

```
text_sanitizer("  HELLO   WORLD  ", "hello")  # ("hello world", 1)
text_sanitizer("Python IS great",   "python") # ("python is great", 1)
text_sanitizer("  CAT  cat  CAT  ", "cat")    # ("cat cat cat", 3)
```

Completion Criteria

Reference Solution: Text Sanitizer

```python
def text_sanitizer(sentence, keyword):
    # Normalize: strip and lowercase
    cleaned = sentence.strip().lower()

    # Collapse multiple spaces
    while "  " in cleaned:
        cleaned = cleaned.replace("  ", " ")

    # Normalize keyword, then count
    keyword_normalized = keyword.lower()
    count = cleaned.count(keyword_normalized)

    return cleaned, count
```

> **Key insight:** Normalize the keyword too — not just the sentence.

> *Source:* `Day2_Hour2_Basics.md` *— Lab Solution*

Common Pitfalls (Hour 6)

- ⚠️ **Forgetting to assign the result** — `msg.lower()` without assignment does nothing
- ⚠️ **Using `find()` as a boolean** — `0` is falsy; always compare `!= -1`
- ⚠️ **Single `replace("  ", " ")` doesn't catch all spaces** — use a `while` loop
- ⚠️ **Forgetting to normalize the keyword** before `.count()`

Quick Check (Hour 6)

1. Why does `message.upper()` not change `message`?
2. What does `.find("x")` return when `"x"` is not in the string?
3. Which method would you use to remove tabs from the start of a line?
4. What is the difference between `.strip()` and `.replace(" ", "")`?

Answers: immutability · `-1` · `.lstrip()` · strip removes only edges; replace removes all

Hour 7: Interactive Input + Type Conversion

Learning Outcomes

- Use `input()` to collect user data at runtime
- Apply `int()`, `float()`, and `str()` to convert text to numeric types
- Combine input and conversion for practical calculations
- Recognize `ValueError` and understand when it occurs
- Build interactive programs with clean, formatted output

> *Source: `Basics/lessons/lecture/Day2_Hour3_Basics.md` — Learning Outcomes*

`input()` Always Returns a String

Every User Response Is Text — Even Digits

```
name = input("Enter your name: ")
print(f"Hello, {name}!")
print(type(name))       # <class 'str'>
```

Even Numeric-Looking Input Is a String

```
age = input("Enter your age: ")
print(type(age))        # <class 'str'>  — not int!
# age + 1               # ❌ TypeError!
```

> "Python cannot guess whether `'25'` is an age, a count, or a product ID. Forcing you to specify is a feature, not a bug."

> *Source: `Day2_Hour3_Basics.md` — Section 3*

Why Type Conversion Is Required

Mixing Types Causes Errors

```
age = input("Enter your age: ")   # returns "25" (string)
# next_year = age + 1             # ❌ TypeError: can only concatenate str to str
```

The Fix: Convert Before Arithmetic

```
age = int(input("Enter your age: "))
next_year = age + 1
print(f"Next year you will be {next_year}")
```

> **Rule:** Collect as text → convert immediately → compute safely.

> *Source:* `Day2_Hour3_Basics.md` *— Section 3*

Conversion Functions

The Three You Need

Function	Converts to	Example
`int(x)`	Whole number	`int("25")` → `25`
`float(x)`	Decimal number	`float("19.99")` → `19.99`
`str(x)`	Text	`str(42)` → `"42"`

Usage

```
age   = int("25")         # 26 after age + 1
price = float("19.99")    # 39.98 after price * 2
label = str(42)           # "42!" after label + "!"
```

> Choose based on domain: count/ID → `int`; measurement/price → `float`

Combining `input()` with Conversion

The Common Pattern

```
age         = int(input("Enter your age: "))
temperature = float(input("Temperature in °F: "))
count       = int(input("How many? "))
```

One-Liner: Collect + Convert in a Single Step

The outer call converts immediately — no separate assignment needed.

```
miles = float(input("Enter distance in miles: "))
km    = miles * 1.60934
print(f"{miles} miles = {km:.2f} km")
```

> *Source: `Day2_Hour3_Basics.md` — Section 3*

`ValueError` — Know It, Don't Fear It

When Conversion Fails

```
# int("hello")  # ValueError: invalid literal for int()
# int("3.5")    # ValueError: invalid literal for int() — has decimal!
# float("abc")  # ValueError: could not convert string to float
```

The Two-Step Fix for Decimals

```
# Option 1: use float if decimals are possible
value = float("3.5")       # 3.5 ✓

# Option 2: float first, then int to truncate
value = int(float("3.5"))  # 3 ✓
```

> Today we use valid (happy-path) inputs. Formal error handling (`try/except`) comes in a later session.

Demo: Temperature Converter

```
fahrenheit_text = input("Enter temperature in °F: ")
print(f"Raw input: {fahrenheit_text}")
print(f"Raw input type: {type(fahrenheit_text)}")   # <class 'str'>

fahrenheit = float(fahrenheit_text)
print(f"Converted type: {type(fahrenheit)}")         # <class 'float'>

celsius = (fahrenheit - 32) * 5 / 9

print(f"\n{fahrenheit:.1f}°F = {celsius:.1f}°C")
```

Test with `98.6`:

```
Raw input type: <class 'str'>
Converted type: <class 'float'>

98.6°F = 37.0°C
```

Lab: Unit Converter

Time: 25–35 minutes

Build one complete converter (your choice):

Option A — Miles → Kilometers

```
=== Miles to Kilometers Converter ===
Enter distance in miles: 10
10.0 miles = 16.09 kilometers
```

Option B — Fahrenheit → Celsius

```
=== Temperature Converter ===
Enter temperature in °F: 98.6
98.6°F = 37.0°C
```

Workflow: Collect → Convert → Compute → Format → Verify

Reference Solutions: Unit Converter

Miles → Kilometers

```
print("=== Miles to Kilometers Converter ===")
miles = float(input("Enter distance in miles: "))
km    = miles * 1.60934
print(f"{miles:.1f} miles = {km:.2f} kilometers")
```

Fahrenheit → Celsius

```
print("=== Temperature Converter ===")
fahrenheit = float(input("Enter temperature in °F: "))
celsius    = (fahrenheit - 32) * 5 / 9
print(f"{fahrenheit:.1f}°F = {celsius:.1f}°C")
```

> *Source:* `Day2_Hour3_Basics.md` *— Section 5, Solution Guidance*

Common Pitfalls (Hour 7)

⚠️ **Forgetting conversion** — `price * 3` on a string repeats text, not math

⚠️ **Using `int()` on a decimal string** — `int("3.5")` raises `ValueError`; use `float()` first

⚠️ **Wrong formula constants** — always plausibility-check (10 miles ≈ 16 km, not 1609)

⚠️ **No units in output** — `16.09` is ambiguous; always include labels

⚠️ **Ugly floating-point output** — use `:.1f` or `:.2f` in f-strings

Quick Check (Hour 7)

1. What type does `input()` always return?
2. Why can't you do `age + 1` right after `input()`?
3. Which function converts `"19.99"` to a decimal number?
4. What error appears when `int()` receives `"3.5"`?

Answers: `str` · still a string · `float()` · `ValueError`

Hour 8: Checkpoint 1 — Fundamentals Mini-Assessment

Learning Outcomes

- Synthesize variables, types, operators, strings, and input/output in one program
- Apply type conversion correctly for real-world data
- Debug and build a multi-step program from requirements
- Explain variable naming and type-choice decisions
- Build a solid foundation for Day 3 (conditionals and loops)

> *Source:* `Basics/lessons/lecture/Day2_Hour4_Basics.md` *— Learning Outcomes*

Checkpoint Format

What This Is

- **Open-book and individual** — notes, previous scripts, and class materials are allowed
- **Timeboxed** — 50 minutes to build and test a complete program
- **Purpose:** demonstrate basic competence, not memorize commands

What It Tests

Skill	Evidence
Input + conversion	`input()` + `int()` / `float()`
Variable naming	Descriptive, snake_case names
Arithmetic	`quantity × price`

Problem Statement: Simple Receipt Generator

Build `checkpoint1_receipt.py` that:

1. Prints a title line
2. Asks for: item name, quantity, price per item
3. Computes **subtotal** = `quantity × price_per_item`
4. Sets **total** = subtotal *(baseline: no tax or discount)*
5. Prints a formatted receipt with all values to 2 decimal places

Expected Output

```
=== Receipt Generator ===
Enter item name: Widget
Enter quantity: 3
Enter price per item: 9.99

========== RECEIPT ==========
```

Demo Walk-Through

Step 1: Collect Input

```
print("=== Receipt Generator ===")

item_name      = input("Enter item name: ").strip()
quantity       = int(input("Enter quantity: "))
price_per_item = float(input("Enter price per item: "))
```

Step 2: Compute

```
subtotal = quantity * price_per_item
total    = subtotal   # baseline: no tax/discount
```

Step 3: Format and Print

```
print()
print("========== RECEIPT ==========")
```

Assessment Rubric

Category	Points	What to Check
Program runs end-to-end	35	No crash on valid input; prompts and output in logical order
Correct math and types	35	`quantity` as `int`, `price` as `float`, subtotal = qty × price, total = subtotal
Readable output formatting	20	Money values to 2 decimals with `:.2f`, clear labels, separators
Code clarity	10	Descriptive names (`subtotal` not `c`), logical flow
Total	**100**	

> **Priority order:** correctness → types → formatting → clarity

> *Source: `Day2_Hour4_Basics.md` — Assessment Rubric*

Common Pitfalls (Checkpoint 1)

⚠️ **Forgetting conversion** — `price * 3` on a string gives `"9.999.999.99"` not `29.97`

⚠️ **`quantity` as `float`** — use `int` for counts; `float(3)` gives `3.0` which obscures intent

⚠️ **`total ≠ subtotal`** — don't add unintended math; baseline has no tax or discount

⚠️ **Missing `:.2f`** — floating-point precision: `29.970000000001` instead of `29.97`

⚠️ **Vague variable names** — `a`, `b`, `c` obscure intent; use `quantity`, `price_per_item`, `subtotal`

> *Source: `Day2_Hour4_Basics.md` — Troubleshooting Pitfalls*

Debrief: Quick-Check Questions

1. **What type does `input()` return?**
   → `str` — always, even for digits

2. **Why use `int` for quantity and `float` for price?**
   → Quantity is a count (whole number); price includes decimals — types clarify intent

3. **How do you format money to 2 decimal places?**
   → `f"${subtotal:.2f}"` in an f-string

4. **What happens if you multiply a string by a number?**
   → String repetition: `"abc" * 3` → `"abcabcabc"`

5. **Why is `total == subtotal` in this checkpoint?**
   → No tax or discount in the baseline version — by design

Session 2 Wrap-Up

What We Covered Today

Hour 5: String Indexing + Slicing

- 0-based indexing, negative indexing
- Slice syntax `s[start:end]` (end exclusive)
- `len()` and `in` for safe, readable code

Hour 6: String Methods

- Immutability: always assign the return value
- `.lower()`, `.strip()`, `.replace()`, `.find()`, `.count()`
- Method chaining for clean pipelines

Hour 7: Input + Type Conversion

Scope Guardrail Reminder

Stay in Basics Scope ✓

✓ String indexing, slicing, and built-in methods
✓ `input()` with explicit type conversion
✓ Clear variable names and formatted output
✓ Happy-path inputs (no `try/except` yet)

Not Yet (Advanced Topics) ✗

✗ Regular expressions
✗ `try/except` error handling
✗ List comprehensions
✗ Lambda functions or decorators

Next Session Preview

Session 3 (Hours 9–12)

- Hour 9: Comparisons and boolean logic
- Hour 10: String formatting with f-strings (deeper dive)
- Hour 11: Text operations with `split()` and `join()`
- Hour 12: Debugging habits at Basics level

> Keep your Checkpoint 1 script — it is valuable practice material for Day 3!

Questions?

Remember:

- Save your work frequently
- Normalize before you search (`lower()` + `strip()`)
- Convert before you calculate (`int()` / `float()`)
- Format your output for humans — include units and labels
- Build incrementally: write 5 lines, run, test, repeat

Thank You!

See you in Session 3!